

PROYECTO 1

Jugador Autónomo para el Juego HEX

Estrategia basada en MC-RAVE con MAST, LGR y Union-Find Incremental

Documento de Estrategia Técnica

AUTORA

Alina María de la Noval Armenteros

Grupo C-311

Algoritmo : MC-RAVE (Gelly & Silver, 2011) + MAST + LGR

Framework : Monte Carlo Tree Search

Tablero : $N \times N$ hexagonal (even-r offset)

Límite de tiempo : 4,5 s por jugada

Índice

1	Introducción	2
2	Monte Carlo Tree Search	2
2.1	Descripción general	2
2.2	Selección — UCT	3
2.3	Expansión	3
2.4	Simulación — política de rollout con MAST, LGR y puentes	4
2.5	Retropropagación	4
3	RAVE y MC-RAVE	4
3.1	Motivación: escasez de datos en las fases iniciales	4
3.2	Estadísticas AMAF	5
3.3	MC-RAVE: combinación de UCT y AMAF	5
4	Política de simulación enriquecida	6
4.1	MAST: Move-Average Sampling Technique	6
4.2	LGR: Last Good Reply	7
4.3	Respuesta de puente (<i>bridge response</i>)	7
4.4	Cadena de prioridades del rollout	8
5	Estrategia del Jugador Implementado	8
5.1	Arquitectura general	8
5.2	Union-Find incremental con nodos virtuales	9
5.3	Implementación MC-RAVE en MCTSNode	10
5.4	Restricción step-2: coherencia de las estadísticas AMAF	10
6	Estrategia Final del Jugador	11
6.1	Diagrama de flujo de <code>SmartPlayer.play()</code>	12
7	Conclusión	12

1 Introducción

El juego **HEX** es un juego de estrategia de dos jugadores en un tablero hexagonal de tamaño $N \times N$. El Jugador 1 (**rojo**) debe conectar el borde izquierdo con el derecho, mientras que el Jugador 2 (**azul**) debe conectar el borde superior con el inferior. Los jugadores alternan turnos colocando una ficha en cualquier celda vacía, y el primero en establecer una ruta continua entre sus bordes gana. El juego no puede terminar en empate [5].

Desde el punto de vista computacional, HEX presenta varias dificultades:

- **Espacio de estados exponencial.** Un tablero de 11×11 tiene $121!$ órdenes posibles de juego.
- **Sin función heurística fiable.** A diferencia del ajedrez, no existen evaluadores de posición bien establecidos.
- **Ramificación alta.** El factor de ramificación disminuye con el avance de la partida, pero es máximo al inicio, lo que hace inviable la búsqueda exhaustiva.

Por estas razones, los métodos de **Monte Carlo Tree Search** (MCTS) son especialmente adecuados para HEX. No requieren una función heurística elaborada, son *anytime* (producen una respuesta en cualquier momento), y su precisión mejora con el tiempo disponible [5]. De hecho, el programa MOHEX, basado en MCTS con RAVE, se convirtió en campeón mundial de HEX computacional en 2009 [5].

La estrategia implementada en este proyecto combina MC-RAVE con un Union-Find incremental para detección eficiente de victoria, y enriquece la política de simulación (*rollout*) mediante tres mecanismos: **MAST** (*Move-Average Sampling Technique*), **LGR** (*Last Good Reply*) y un detector local de **puentes** (*bridge response*). Estos mecanismos dotan al agente de una política de simulación significativamente más informada que el muestreo uniforme estándar, siguiendo la línea de MoHex [1] y su sucesor MoHex 2.0 [2], los programas de referencia en MCTS para HEX.

2 Monte Carlo Tree Search

2.1 Descripción general

Monte Carlo Tree Search es un método de búsqueda que combina la estructura de un árbol de decisión con muestreo aleatorio de resultados [5]. El algoritmo itera sobre cuatro fases hasta agotar el presupuesto computacional, construyendo progresivamente un árbol sesgado hacia las regiones del espacio de búsqueda más prometedoras.

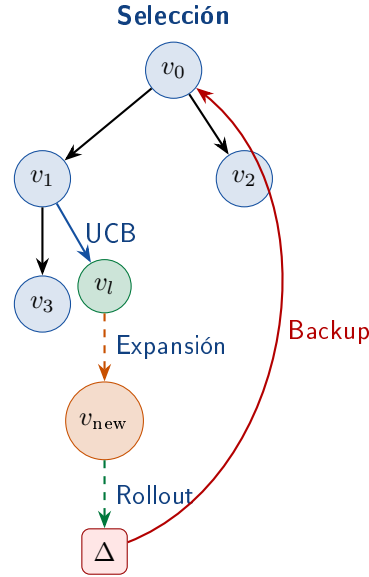
Algoritmo 1: MCTS — Iteración general (Browne et al. [5])

Input: Estado raíz s_0
Output: Acción óptima a^*

```

1 Crear nodo raíz  $v_0$  con estado  $s_0$ 
2 while dentro del presupuesto computacional do
3    $v_l \leftarrow \text{TreePolicy}(v_0)$  // Selección + Expansión
4    $\Delta \leftarrow \text{Rollout}(s(v_l))$  // Simulación con MAST+LGR+puente
5    $\text{Backup}(v_l, \Delta)$  // Retropropagación + actualización MAST/LGR
6 end
7 return  $a(\text{BestChild}(v_0, \theta))$ 
```

La figura siguiente ilustra las cuatro fases en una sola iteración:



2.2 Selección — UCT

Durante la fase de selección, el árbol se recorre desde la raíz hasta alcanzar un nodo con movimientos no explorados, eligiendo en cada nivel el hijo con mayor valor de acuerdo a la política UCB1 (*Upper Confidence Bound*) [5].

Definición: UCT — selección en árbol (Gelly & Silver [4]; Browne et al. [5])

Para un nodo v con padre $\text{padre}(v)$, el valor UCT se define como:

$$\text{UCT}(v) = \underbrace{\frac{Q(v)}{N(v)}}_{\text{explotación}} + C_p \underbrace{\sqrt{\frac{\ln N(\text{padre}(v))}{N(v)}}}_{\text{exploración}} \quad (1)$$

donde:

- $Q(v)$ — suma acumulada de victorias en el nodo v ; $Q(v)/N(v)$ es la tasa de victorias.
- $N(v)$ — número de visitas al nodo v .
- $N(\text{padre}(v))$ — número de visitas al padre de v .
- $C_p \geq 0$ — constante de exploración (`EXPLORATION_C` en el código).

El primer término favorece la **explotación**: los movimientos con mayor tasa de victorias. El segundo término favorece la **exploración**: su valor decrece a medida que $N(v)$ crece, incentivando la visita a nodos poco explorados. Cuando $N(v) = 0$ el valor es $+\infty$, garantizando que todo hijo sea visitado al menos una vez [5].

En la implementación, `EXPLORATION_C` = 0.0, es decir, el término de exploración UCT está desactivado. Browne et al. señalan explícitamente que, para Go y HEX, cuando se emplean heurísticas de historia como RAVE y AMAF, se ha observado que esta constante debe ser cero [5], pues las estadísticas AMAF ya proveen exploración implícita. Este punto se retoma en la Sección 3.3.

2.3 Expansión

Cuando el algoritmo de selección alcanza un nodo v que no está *completamente expandido* (es decir, tiene al menos un movimiento no explorado), se selecciona uno de esos movimientos sin

probar y se añade un nuevo hijo v' al árbol:

$$s(v') = f(s(v), a), \quad a(v') = a.$$

Cada nodo hijo hereda del padre la lista de celdas vacías y una copia del estado Union-Find actualizada con el movimiento recién expandido, de modo que el estado de conectividad de cada nodo del árbol es consistente con la secuencia de movimientos que conduce a él desde la raíz.

2.4 Simulación — política de rollout con MAST, LGR y puentes

A partir del nodo recién expandido, se ejecuta una simulación hasta alcanzar un estado terminal. La simulación es el método `rollout()` de la clase `MCTSTree`, que implementa una política informada en tres niveles de prioridad decreciente, en lugar de un muestreo uniforme. El detalle completo de esta política se describe en la Sección 4.

En términos estructurales, el rollout opera de la siguiente forma:

1. Se realizan copias profundas de las dos estructuras Union-Find del nodo (`node_uf1`, `node_uf2`) para no contaminar el árbol.
2. Se mantiene un conjunto `legal` con las celdas vacías restantes.
3. En cada turno se selecciona un movimiento según la cadena de prioridades (puente $\rightarrow$ LGR $\rightarrow$ MAST), se actualiza el tablero y el Union-Find, y se comprueba de inmediato si hay victoria.
4. Se devuelve el ganador $w \in \{1, 2\}$ y el diccionario `moves_by_player` con todos los movimientos realizados por cada jugador durante la simulación, necesario para actualizar las estadísticas AMAF, MAST y LGR en la retropropagación.

2.5 Retropropagación

Una vez obtenido el ganador w del rollout, se actualizan las estadísticas de todos los nodos en el camino desde la hoja hasta la raíz:

$$N(v) \leftarrow N(v) + 1, \tag{2}$$

$$Q(v) \leftarrow Q(v) + \mathbf{1}[\text{player}(v) = w], \tag{3}$$

donde $\mathbf{1}[\cdot]$ es la función indicadora: se suma 1 a $Q(v)$ únicamente si el jugador que realizó el movimiento en el nodo v es el ganador w .

Perspectiva *player-centric*. Cada nodo almacena el identificador del jugador que realizó el movimiento (`node.player`). Esto simplifica la retropropagación: no es necesario invertir el signo de la recompensa al subir por el árbol, ya que cada nodo acumula victorias solo desde su propia perspectiva. El nodo raíz tiene `player=None` y por tanto nunca acumula victorias —solo visitas.

Además de actualizar las estadísticas UCT y AMAF, la retropropagación actualiza las estadísticas **MAST** y la tabla **LGR**, como se detalla en las secciones 4.1 y 4.2.

3 RAVE y MC-RAVE

3.1 Motivación: escasez de datos en las fases iniciales

Un problema central de MCTS puro es que las estadísticas de los nodos cercanos a la raíz son escasas durante las primeras iteraciones. Un movimiento puede no haberse visitado lo suficiente para que su estimación $Q(v)/N(v)$ sea fiable, lo que conduce a decisiones subóptimas al inicio de la búsqueda.

El algoritmo **RAVE** (*Rapid Action Value Estimation*) [4] propone aprovechar la heurística AMAF (*All Moves As First*): si una acción a apareció en alguna simulación posterior al estado s , esa información también es relevante para estimar el valor de a en s , independientemente del turno exacto en que se jugó.

3.2 Estadísticas AMAF

Definición: valor AMAF

El valor AMAF de la acción a en el estado s se define como:

$$\tilde{Q}(s, a) = \frac{\tilde{W}(s, a)}{\tilde{N}(s, a)} \quad (4)$$

donde:

- $\tilde{W}(s, a)$ — número de victorias en simulaciones donde a fue jugado en cualquier turno posterior a s .
- $\tilde{N}(s, a)$ — número de simulaciones donde a fue jugado en cualquier turno posterior a s .

La hipótesis subyacente es que, en juegos de posición como HEX, el valor de una jugada no depende fuertemente del turno exacto en que se realiza [4]. Las estadísticas AMAF acumulan evidencia mucho más rápido que las estadísticas UCT clásicas, aunque introducen un sesgo sistemático.

3.3 MC-RAVE: combinación de UCT y AMAF

El algoritmo **MC-RAVE** [4] combina el valor UCT $Q(s, a)$ con el valor AMAF $\tilde{Q}(s, a)$ mediante una ponderación que varía dinámicamente con el número de visitas:

Definición: valor combinado MC-RAVE (Gelly & Silver [4])

$$Q^*(s, a) = (1 - \beta) Q(s, a) + \beta \tilde{Q}(s, a) \quad (5)$$

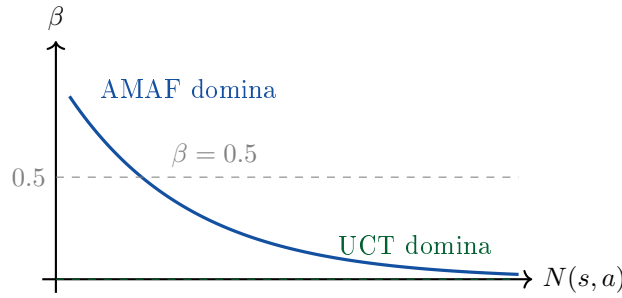
con la ponderación mínima-MSE:

$$\beta(s, a) = \frac{\tilde{N}(s, a)}{N(s, a) + \tilde{N}(s, a) + 4 N(s, a) \tilde{N}(s, a) b^2} \quad (6)$$

donde b es el parámetro de sesgo RAVE (RAVE_BIAS en el código).

Comportamiento dinámico de β . Cuando $N(s, a)$ y $\tilde{N}(s, a)$ son pequeños (inicio de la búsqueda), $\beta \approx 1$ y el valor combinado está dominado por la estimación AMAF, que es menos precisa pero tiene mucha más evidencia. A medida que el árbol crece y $N(s, a) \rightarrow \infty$, el denominador de (6) crece cuadráticamente y $\beta \rightarrow 0$, de modo que el valor UCT —más preciso pero más lento— termina dominando. Este mecanismo garantiza que la convergencia asintótica de MC-RAVE sea equivalente a la de UCT [4].

Parámetro b y C_p . El parámetro b en (6) se relaciona con el parámetro de equivalencia k de Gelly & Silver mediante $k = 1/(4b^2)$ [4]. En la implementación se usa $b = 0,00913$, equivalente a $k \approx 3000$, determinado empíricamente para tableros de 11×11 [4]. Se establece `EXPLORATION_C` = 0.0 en consonancia con Browne et al. [5].



4 Política de simulación enriquecida

La política de rollout es uno de los componentes más importantes del jugador implementado. En lugar de muestrear los movimientos de forma uniforme aleatoria —práctica habitual en MCTS básico—, el método `rollout()` aplica en cada turno una **cadena de prioridades** que combina tres mecanismos aprendidos o basados en conocimiento local. Esta cadena sigue el orden: (1) respuesta de puente, (2) LGR, (3) MAST.

4.1 MAST: Move-Average Sampling Technique

MAST (*Move-Average Sampling Technique*) es una técnica propuesta por Finnsson & Björnsson [6] para guiar la política de rollout en MCTS sin necesidad de conocimiento específico del dominio elaborado. La idea central es estimar el valor de cada movimiento a para cada jugador p acumulando las recompensas obtenidas cada vez que a fue jugado por p durante alguna simulación, independientemente del estado exacto del tablero.

Definición: valor MAST de un movimiento

Sea $\mathcal{T}_p(a)$ el número de veces que el jugador p ha jugado el movimiento a durante simulaciones pasadas, y $\mathcal{W}_p(a)$ el número de victorias obtenidas por p en esas simulaciones. El **valor MAST** del par (p, a) es:

$$\hat{Q}_p(a) = \begin{cases} \frac{\mathcal{W}_p(a)}{\mathcal{T}_p(a)} & \text{si } \mathcal{T}_p(a) > 0, \\ 0,5 & \text{en caso contrario (valor neutro por defecto).} \end{cases} \quad (7)$$

Para seleccionar el siguiente movimiento se emplea **muestreo de Boltzmann** (también conocido como *softmax*) [5], que asigna probabilidad proporcional a $\exp(\hat{Q}_p(a)/\tau)$ a cada movimiento legal a :

$$P_p(a \mid \text{legal}) = \frac{\exp(\hat{Q}_p(a)/\tau)}{\sum_{a' \in \text{legal}} \exp(\hat{Q}_p(a')/\tau)} \quad (8)$$

donde $\tau > 0$ es la **temperatura de MAST** (`MAST_TEMPERATURE = 0,3` en el código). Un valor de τ pequeño concentra la distribución alrededor del movimiento con mayor $\hat{Q}_p$, mientras que $\tau \rightarrow \infty$ recupera el muestreo uniforme. La implementación del método `_mast_pick()` calcula estos pesos por lista y realiza muestreo por ruleta en tiempo $O(|\text{legal}|)$.

Indexación por jugador. Un detalle clave de la implementación es que las estadísticas MAST se indexan por el par `(player_id, move)`, no solo por el movimiento. Esto evita la *contaminación cruzada* entre jugadores: el valor MAST de un movimiento para el Jugador 1 es independiente de su valor para el Jugador 2. Esta separación es correcta porque el valor de un movimiento

en HEX depende críticamente de qué jugador lo realiza (conectar borde izquierdo-derecho vs. superior-inferior).

Actualización en la retropropagación. Al finalizar cada simulación, las estadísticas MAST de todos los movimientos realizados se actualizan en `_backpropagate()`:

$$\mathcal{T}_p(a) \leftarrow \mathcal{T}_p(a) + 1, \quad (9)$$

$$\mathcal{W}_p(a) \leftarrow \mathcal{W}_p(a) + \mathbf{1}[p = w], \quad (10)$$

donde w es el ganador de la simulación. Las tablas `mast_total` y `mast_wins` residen en `MCTSTree` (nivel global del motor), por lo que se acumulan a lo largo de todas las iteraciones de la búsqueda.

4.2 LGR: Last Good Reply

LGR (*Last Good Reply*) es una heurística de respuesta propuesta por Baier & Drake [7] para enriquecer la política de rollout en juegos de dos jugadores. La idea es almacenar, para cada jugador p y cada movimiento m_{opp} del oponente, la respuesta m_{reply} que resultó beneficiosa para p en simulaciones pasadas donde el oponente jugó m_{opp} .

Definición: tabla LGR

La tabla LGR de un jugador p es una función parcial:

$$\text{LGR}_p : \mathcal{M}_{\text{opp}} \rightarrow \mathcal{M}_p \quad (11)$$

donde $\mathcal{M}_{\text{opp}}$ es el conjunto de movimientos del oponente y $\mathcal{M}_p$ el del jugador p . Durante el rollout, si el oponente acaba de jugar m_{opp} y $\text{LGR}_p(m_{\text{opp}})$ está definido y la celda correspondiente sigue vacía, se juega esa respuesta con probabilidad $1 - \varepsilon$ (`LGR_RANDOMNESS = 0,2`).

Actualización de la tabla LGR. La tabla se actualiza al final de cada simulación, pero solo con los movimientos del *ganador*, pues se desea aprender respuestas que conduzcan a victoria. Sean $S_1 = [m_1^{(1)}, m_1^{(2)}, \dots]$ y $S_2 = [m_2^{(1)}, m_2^{(2)}, \dots]$ las secuencias de movimientos realizados por el Jugador 1 y el Jugador 2 respectivamente durante el rollout. Si el ganador es el Jugador 1:

$$\text{LGR}_1[m_2^{(i)}] \leftarrow m_1^{(i+\delta)}, \quad \delta \in \{0, 1\} \quad (12)$$

donde δ es un desplazamiento que corrige el orden de turno según quién mueve primero. De forma análoga para el Jugador 2 cuando gana. La semántica de la tabla es: “si el oponente jugó en la celda m_{opp} , respondo en la celda $\text{LGR}_p[m_{\text{opp}}]$ ”.

Exploración estocástica. La consulta LGR es probabilística: con probabilidad `LGR_RANDOMNESS = 0,2`, se ignora la respuesta almacenada aunque esté disponible y se delega la selección al siguiente nivel de prioridad (MAST). Esto evita que la política de rollout quede completamente determinada por la tabla LGR, manteniendo diversidad en las simulaciones.

4.3 Respuesta de puente (*bridge response*)

Un **puente** es la conexión virtual más elemental en HEX [8]: dos fichas propias no adyacentes cuya única separación son exactamente dos celdas vacías compartidas como vecinas. La propiedad clave es que un puente es *indestructible* con respuesta correcta: si el oponente ocupa una de las dos celdas vacías del puente, el jugador siempre puede ocupar la otra y mantener la conectividad garantizada entre ambas fichas. Por ello, MoHex [1] y MoHex 2.0 [2] incorporan la respuesta

de puente —denominada *savebridge*— como el único patrón de conocimiento de dominio en su política de rollout.

Definición: puente y portador (Anshelevich [8]; Arneson et al. [1])

Sean c_1, c_2 dos celdas ocupadas por el mismo jugador p y sea $\mathcal{N}(c)$ el conjunto de vecinas hexagonales de c . El par (c_1, c_2) forma un **puente** si y solo si:

$$\mathcal{N}(c_1) \cap \mathcal{N}(c_2) = \{\gamma_1, \gamma_2\}, \quad \gamma_1, \gamma_2 \notin \{\text{celdas ocupadas}\}.$$

El conjunto $\Gamma = \{\gamma_1, \gamma_2\}$ es el **portador** (*carrier*) del puente. Si el oponente juega en γ_i , el jugador p responde en γ_{3-i} , preservando la conectividad entre c_1 y c_2 .

Durante el rollout, el método `_bridge_response()` se ejecuta justo después de cada movimiento del oponente en la celda m_{opp} : para cada par de fichas propias que sean vecinas de m_{opp} , se comprueba si existe una portadora común aún libre; de encontrarse, esa celda se retorna como movimiento. Si no hay ningún puente amenazado, la función devuelve **None** y la decisión pasa al siguiente nivel de la cadena (LGR o MAST). La respuesta de puente tiene la **prioridad más alta** en el rollout, en consonancia con MoHex [1] y MoHex 2.0 [2].

4.4 Cadena de prioridades del rollout

La política completa de selección de movimiento en cada turno del rollout sigue el esquema siguiente:

Política de rollout — cadena de prioridades

Input: Estado s , jugador p , último movimiento del oponente m_{last} , legales

```

1  $mv \leftarrow \text{BridgeResponse}(s, m_{\text{last}}, p, \text{legales})$            // Prioridad 1: preservar
   conexión virtual (savebridge [1, 2])
2 if  $mv = \text{None}$  and  $m_{\text{last}} \neq \text{None}$  then
3    $\text{cand} \leftarrow \text{LGR}_p[m_{\text{last}}]$                                // Prioridad 2: LGR
4   if  $\text{cand} \neq \text{None}$  and  $\text{cand} \in \text{legales}$  then
5     if  $\text{Uniform}(0, 1) > \varepsilon$  then
6        $mv \leftarrow \text{cand}$ 
7 if  $mv = \text{None}$  then
8    $mv \leftarrow \text{MastPick}(\text{legales}, p)$                        // Prioridad 3: muestreo Boltzmann
9 return  $mv$ 

```

Este diseño en cascada garantiza que el conocimiento más específico (la preservación de conexiones virtuales) tiene precedencia, mientras que el conocimiento estadístico global (MAST) actúa como política de base robusta cuando ninguna respuesta táctica es aplicable.

5 Estrategia del Jugador Implementado

5.1 Arquitectura general

La implementación se organiza en tres componentes principales, cuya relación se muestra en la Figura 1.

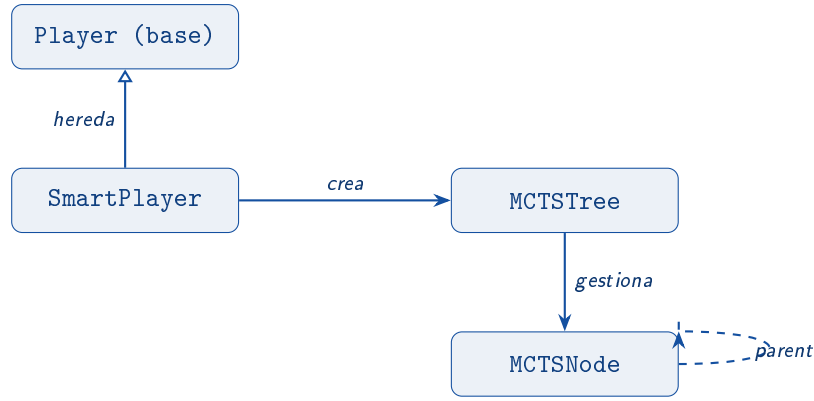


Figura 1: Relación entre las clases de la implementación.

SmartPlayer. Clase principal que implementa la interfaz **Player**. Encapsula la lógica de decisión en cascada: evalúa movimientos decisivos mediante Union-Find (`_find_decisive_moves`) y, si no aplica ninguna regla directa, delega toda la búsqueda al motor **MCTSTree**.

MCTSTree. Motor de búsqueda MC-RAVE. Mantiene la raíz del árbol, las tablas globales MAST (`mast_wins`, `mast_total`) y LGR (`lgr`), gestiona el presupuesto de tiempo y ejecuta el bucle de iteraciones. Internamente, cada iteración invoca `rollout()`, que utiliza las copias Union-Find del nodo expandido para detectar victorias durante la simulación de forma eficiente en cada paso.

MCTSNode. Nodo individual del árbol. Almacena las estadísticas UCT (`visits`, `wins`) y AMAF (`visits_amaf`, `wins_amaf`), la lista incremental de celdas vacías (`empty`), los movimientos sin explorar (`untried_moves`), el puntero al nodo padre (`parent`), y las dos estructuras Union-Find (`uf1`, `uf2`) que representan el estado de conexiones de cada jugador en ese nodo.

5.2 Union-Find incremental con nodos virtuales

Una optimización crítica de la implementación es el uso de Union-Find (DSU) incremental basado en las técnicas de Tarjan [9] (path halving + union by rank) y la idea de nodos virtuales de borde de Anshelevich [8].

La clase `HexUnionFind` trabaja con índices lineales. Dado un tablero de tamaño `size`, se definen `n = size * size` nodos reales y cuatro **nodos virtuales** adicionales que representan los bordes del tablero:

Variable	Índice	Representa	Jugador
VL	<code>n</code>	Borde izquierdo (columna 0)	1
VR	<code>n+1</code>	Borde derecho (columna <code>size-1</code>)	1
VT	<code>n+2</code>	Borde superior (fila 0)	2
VB	<code>n+3</code>	Borde inferior (fila <code>size-1</code>)	2

Cuando se coloca una ficha del Jugador 1 en la columna 0, se ejecuta `union(i, VL)`; en la columna `size-1`, `union(i, VR)`. La condición de victoria del Jugador 1 se reduce entonces a comprobar si `find(VL) == find(VR)`, operación de coste amortizado prácticamente constante gracias a la compresión de caminos y la unión por rango [9]. Análogamente para el Jugador 2 con VT y VB.

Propagación incremental a nodos hijo. Al crear un nuevo nodo hijo en la expansión del árbol, se realiza una copia profunda del estado Union-Find del padre (`copy.deepcopy`) y se aplica `uf.place()` con el movimiento del hijo. De este modo, cada nodo del árbol mantiene su propio estado de conectividad, y el rollout puede continuar desde ese estado sin reconstrucción.

5.3 Implementación MC-RAVE en MCTSNode

Cada nodo almacena cuatro contadores:

Atributo	Tipo	Descripción
<code>visits</code>	<code>int</code>	$N(v)$ — visitas UCT
<code>wins</code>	<code>float</code>	$W(v)$ — victorias UCT
<code>visits_amaf</code>	<code>int</code>	$\tilde{N}(v)$ — visitas AMAF
<code>wins_amaf</code>	<code>float</code>	$\tilde{W}(v)$ — victorias AMAF

El valor de selección se calcula en el método `rave_value()`, que implementa la ecuación (5):

Cálculo de `rave_value()` en MCTSNode

```

1 if visits == 0 then
2   if visits_amaf > 0 then
3     return  $\tilde{W}/\tilde{N}$  + FPU           // FPU = 0.35, First Play Urgency
4   return + $\infty$                  // Nodo no visitado: prioridad máxima
5  $q_{uct} \leftarrow W/N$ 
6 if visits_amaf > 0 then
7    $q_{amaf} \leftarrow \tilde{W}/\tilde{N}$ 
8    $\beta \leftarrow \tilde{N}/(N + \tilde{N} + 4 N \tilde{N} b^2)$ 
9    $q \leftarrow (1 - \beta) q_{uct} + \beta q_{amaf}$ 
10 else
11    $q \leftarrow q_{uct}$ 
12 return  $q + C_p \sqrt{\ln N_{padre}/N}$ 

```

El parámetro FPU (*First Play Urgency*) = 0,35, descrito en Browne et al. [5], asigna un valor provisional a nodos que tienen estadísticas AMAF disponibles pero todavía no han sido visitados directamente por la política de árbol. Si un nodo no tiene ningún tipo de estadísticas (ni UCT ni AMAF), se le asigna $+\infty$ para garantizar que todo movimiento sea explorado al menos una vez.

5.4 Restricción step-2: coherencia de las estadísticas AMAF

Las estadísticas AMAF *solo se actualizan para movimientos del mismo jugador*. Esto refleja el requisito de Gelly & Silver [4]: la actualización AMAF de (s_t, a_u) se realiza únicamente cuando el turno en u corresponde al mismo jugador que en t .

En la implementación, la retropropagación itera desde la hoja hasta la raíz; en cada nodo con hijos, determina el jugador que jugaría a continuación (`next_player`) y actualiza `visits_amaf` / `wins_amaf` solo en los hijos que corresponden a movimientos realizados por ese jugador durante el rollout:

Retropropagación completa con AMAF, MAST y LGR

Input: Nodo hoja v_ℓ ; ganador w ; $movs_p1$: celdas jugadas por el Jugador 1 en el rollout; $movs_p2$: celdas jugadas por el Jugador 2 en el rollout

```

1  $v \leftarrow v_\ell$ 
2 while  $v \neq None$  do
3    $v.visits += 1$ 
4   if  $v.player = w$  then
5      $v.wins += 1$  // este nodo corresponde al ganador
6   if  $v$  tiene hijos expandidos then
7      $p_{sig} \leftarrow$  jugador que mueve en los hijos de  $v$ 
8      $movs\_sig \leftarrow movs\_p1$  si  $p_{sig} = 1$ , si no  $movs\_p2$ 
9     for cada celda  $m \in movs\_sig$  tal que  $m \in v.children$  do
10      // actualizar estadísticas AMAF del hijo que corresponde a
11      // ese movimiento
12       $v.children[m].visits\_amaf += 1$ 
13      if  $v.children[m].player = w$  then
14         $v.children[m].wins\_amaf += 1$ 
15     $v \leftarrow v.parent$ 

    // Actualización global de MAST: acumular para ambos jugadores
16 for  $(p, m) \in \{(1, movs\_p1), (2, movs\_p2)\}$  do
17    $\mathcal{T}_p(m) += 1$ ; if  $p = w$  then  $\mathcal{W}_p(m) += 1$ 

    // Actualización de LGR: solo con los movimientos del jugador ganador
18 Actualizar  $LGR_w$  emparejando cada jugada del oponente con la respuesta
    ganadora

```

6 Estrategia Final del Jugador

La lógica del método `SmartPlayer.play()` implementa una **estrategia en cascada** que combina reglas deterministas simples con la búsqueda MC-RAVE:

Estrategia en cascada de SmartPlayer

1. **Un solo movimiento legal.** Si solo hay una celda vacía, se juega ahí directamente.
2. **Primer movimiento (tablero vacío).** Se juega en el centro ($\lfloor N/2 \rfloor, \lfloor N/2 \rfloor$), posición estratégicamente ventajosa en HEX.
3. **Movimiento decisivo.** Se comprueba, mediante Union-Find con guardado/restauración, si existe un movimiento que gane la partida inmediatamente. Si existe, se juega ahí.
4. **Movimiento anti-decisivo.** Se comprueba si el oponente puede ganar en su siguiente turno y, en ese caso, se bloquea esa jugada.
5. **MC-RAVE durante 4,5 segundos.** Se ejecuta el motor MC-RAVE con rollout enriquecido (MAST + LGR + puentes). Al finalizar, se selecciona el movimiento más robusto (hijo más visitado).

Selección del hijo más visitado. Tras el presupuesto de búsqueda, la acción seleccionada es la del **hijo más visitado** de la raíz (*robust child* según Browne et al. [5]). Esta elección es más

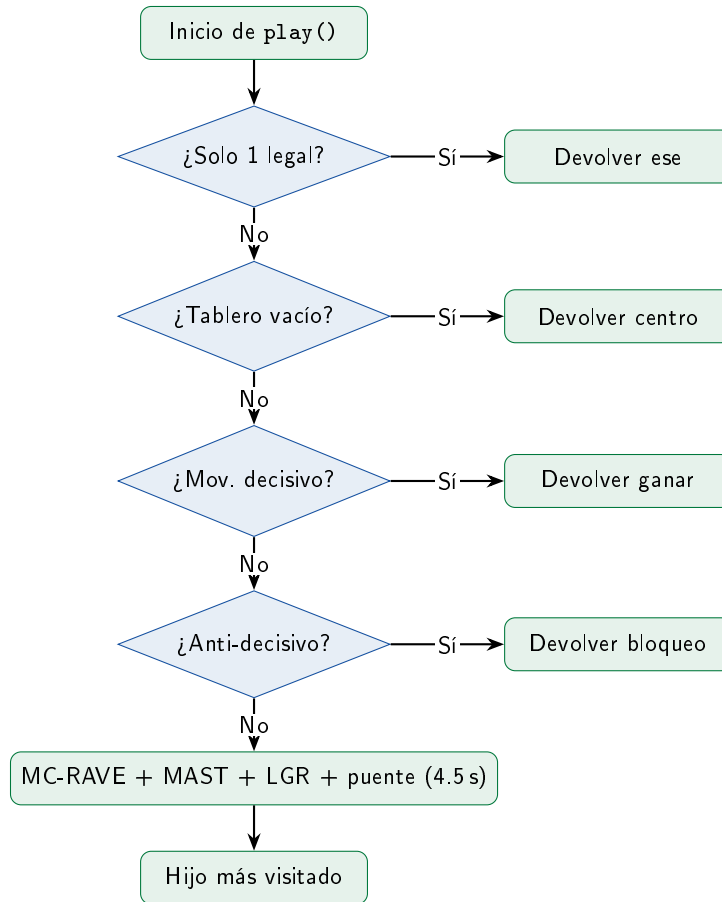
robusta estadísticamente: el hijo más visitado tiene las estimaciones más fiables y concentra la mayor parte de la evidencia de simulación. Formalmente:

$$a^* = \arg \max_{a \in A(s_0)} N(\text{hijo}(v_0, a)).$$

Movimientos decisivos y anti-decisivos. Las comprobaciones de los pasos 3 y 4 usan la función `_find_decisive_moves()`, que recorre cada celda vacía del tablero y simula, mediante el estado Union-Find con guardado y restauración del estado interno, la colocación de la ficha propia o del oponente para determinar si la partida quedaría resuelta en ese turno. Browne et al. [5] clasifican esta técnica bajo el nombre de *decisive moves* dentro de las mejoras de la política de árbol. Su ejecución es determinista y evita por completo la búsqueda MCTS cuando la situación táctica es forzada, bien para ganar de inmediato o bien para impedir una victoria inmediata del oponente.

Gestión del tiempo. El límite de tiempo de 4,5s es inferior al límite de descalificación de 5 s establecido por las reglas del torneo, proporcionando un margen de seguridad para la inicialización del árbol, las comprobaciones de movimientos decisivos/anti-decisivos y la latencia del sistema.

6.1 Diagrama de flujo de `SmartPlayer.play()`



7 Conclusión

Se ha presentado el diseño e implementación de un jugador autónomo para el juego HEX basado en el algoritmo MC-RAVE [4], integrado con un Union-Find incremental y una política de rollout enriquecida mediante MAST [6], LGR [7] y detección local de puentes.

Los elementos principales de la estrategia son:

- **MC-RAVE con $C_p = 0$ y $k \approx 3000$.** La combinación dinámica de estadísticas UCT y AMAF permite aprender más rápido en las fases tempranas sin sacrificar la convergencia asintótica a la política óptima [4].
- **MAST con muestreo de Boltzmann ($\tau = 0,3$).** Las estadísticas de movimiento acumuladas a lo largo de todas las simulaciones guían la política de rollout de forma estadísticamente fundada, indexadas por par (jugador, movimiento) para evitar contaminación cruzada [6].
- **LGR con exploración estocástica ($\varepsilon = 0,2$).** La tabla de respuestas ganadoras permite al rollout explotar patrones de réplica aprendidos en partidas simuladas anteriores [7].
- **Respuesta de puente.** Heurística local de alta prioridad que preserva estructuras de conexión virtualmente indestructibles, imitando el comportamiento de jugadores expertos en HEX.
- **Union-Find con nodos virtuales.** Representación de la conectividad de cada jugador mediante cuatro nodos virtuales de borde, que reduce la detección de victoria a una consulta de representantes sobre la estructura Union-Find [8, 9].
- **Detección de movimientos decisivos.** Evaluación determinista previa a la búsqueda MCTS que identifica jugadas ganadoras o de bloqueo obligatorio mediante el estado Union-Find con restauración, siguiendo la técnica *decisive moves* de Browne et al. [5].
- **Selección del hijo más visitado.** Elección estadísticamente robusta que prefiere la fiabilidad de las estimaciones sobre el valor esperado puntual [5].

La integración de estos mecanismos produce un jugador competitivo dentro del límite de 5 s por jugada establecido en las reglas del torneo, con una política de simulación significativamente más informada que el muestreo uniforme estándar.

Referencias

- [1] B. Arneson, R. B. Hayward, and P. Henderson, Monte Carlo Tree Search in Hex, *IEEE Transactions on Computational Intelligence and AI in Games*, 2(4):251–258, 2010.
- [2] S.-C. Huang, B. Arneson, R. B. Hayward, M. Müller, and J. Pawlewicz, MoHex 2.0: A Pattern-Based MCTS Hex Player, In *Computers and Games*, Lecture Notes in Computer Science, vol. 8427, pp. 60–71. Springer, 2014.
- [3] T. Cazenave and A. Saffidine, Monte-Carlo Hex, In *Proceedings of the AAAI Workshop on Computer Games*, 2006.
- [4] S. Gelly and D. Silver, Monte-Carlo Tree Search and Rapid Action Value Estimation in Computer Go, *Artificial Intelligence*, 175(11):1856–1875, 2011.
- [5] C. Browne, E. Powley, D. Whitehouse, S. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, A Survey of Monte Carlo Tree Search Methods, *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.
- [6] H. Finnsson and Y. Björnsson, Simulation-Based Approach to General Game Playing, In *Proceedings of the 23rd AAAI Conference on Artificial Intelligence*, pp. 259–264, 2008.
- [7] H. Baier and P. D. Drake, The Power of Forgetting: Improving the Last-Good-Reply Policy in Monte Carlo Go, *IEEE Transactions on Computational Intelligence and AI in Games*, 2(4):303–309, 2010.
- [8] V. Anshelevich, A hierarchical approach to computer Hex, *Artificial Intelligence*, 134(1-2):101–120, 2002.
- [9] R. E. Tarjan, Efficiency of a good but not linear set union algorithm, *Journal of the ACM*, 22(2):215–225, 1975.